# Connected Component Analysis for Multi-Vehicle Detection in Shipment Tracking Systems

*Bachelors of Technology*

*in*

*Information Technology*
*Bachelor's of Technology Project Report*

*by*

**Aditya Kumar Singh (2022IMG-005)**

*Under the supervision of*

**Prof. Kiran Kumar Pattanaik**

विश्वजीवनामृतं ज्ञानम्

**ABV INDIAN INSTITUTE OF INFORMATION TECHNOLOGY AND MANAGEMENT**

**GWALIOR-474 015**

# CANDIDATE DECLARATION

I hereby certify that the work, which is being presented in the report, entitled **Connected Component Analysis for Multi-Vehicle Detection in Shipment Tracking Systems**, in fulfillment of the requirement for the award of the Degree of **Bachelor of Technology (B.Tech)** and submitted to the institution is an authentic record of my work carried out during the period *January 2025* to *March 2025* under the supervision of **Prof. Kiran Kumar Pattanaik**. I have also cited the reference to the text(s)/figure(s)/table(s) from where they have been taken.

Date: Signature of the Candidate

This is to confirm that the candidate's above statement is true to the best of my knowledge.

Date: Signature of the Project Supervisor

# ABSTRACT

Shipment tracking systems depend on GPS position data to provide real-time visibility, accurate ETAs, and reliable operational insights. In production environments, this data is often affected by device errors, data integration issues, and system inconsistencies. One critical failure case occurs when a single shipment is incorrectly associated with multiple tracking vehicles, leading to physically impossible movement patterns such as long-distance jumps, oscillating locations, or simultaneous positions in different places. These errors severely impact customer trust, billing accuracy, and operational decision-making.

This report proposes a graph-theoretic framework to detect and validate such multi-vehicle tracking anomalies. Each GPS position is modeled as a node in a directed graph, and edges are created only when movement between two positions is physically feasible within realistic speed and time constraints. The resulting Position Reachability Graph allows shipment trajectories to be analyzed through connectivity rather than ad-hoc rules.

Connected components in the graph represent sets of positions that could belong to a single vehicle. The presence of multiple large, disconnected components indicates concurrent tracking by multiple vehicles. To account for real-world GPS noise, the framework incorporates jitter tolerance, ensuring that minor measurement errors do not cause incorrect fragmentation of valid trajectories.

Legitimate vehicle handovers, such as inter-modal transfers at ports or hubs, are handled using a secondary clustering and merging process based on spatial and temporal proximity. This prevents valid handovers from being misclassified as errors.

The proposed approach is mathematically rigorous, computationally efficient, and suitable for large-scale deployment. By framing shipment validation as a graph connectivity problem, the system provides a robust, explainable, and reliable solution for improving data quality in shipment tracking platforms.

*Keywords:* Shipment Tracking, Multi-Vehicle Detection, Connected Components, Position Reachability Graph, GPS Validation, Graph Theory

# ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Prof. Kiran Kumar Pattanaik for his invaluable guidance, encouragement, and support throughout the course of this project. His insightful feedback, expert advice, and constructive criticism have been instrumental in shaping my understanding of the subject and refining my approach. His constant motivation and belief in my abilities have inspired me to explore new ideas and push my boundaries.

I am also thankful to my institution, ABV-IIITM Gwalior, for providing me with the necessary resources and a conducive learning environment to conduct this research. Additionally, I extend my appreciation to my peers and colleagues for their insightful discussions, constructive feedback, and unwavering support, which have played a significant role in the development of this work.

Finally, I am grateful to my family and friends for their constant encouragement and support, which has helped me stay motivated and focused throughout this journey.

Aditya Kumar Singh

# TABLE OF CONTENTS

**ABSTRACT** ii

**ACKNOWLEDGEMENT** iii

**LIST OF TABLES** v

**LIST OF FIGURES** vi

**1 PROBLEM STATEMENT** 1

1.1 Introduction . . . 1
1.2 Problem: Shipment Tracked by Multiple Vehicles . . . 1
1.2.1 Problem Description . . . 1
1.2.2 Observable Symptoms . . . 2
1.2.3 Legitimate Exception: Vehicle Handovers . . . 2
1.2.4 The Validation Challenge . . . 2
1.2.5 Business Impact . . . 2
1.3 System Overview . . . 3

**2 GRAPH THEORY BACKGROUND** 4

2.1 Graph Theory Fundamentals . . . 4
2.1.1 Basic Definitions . . . 4
2.1.2 Paths and Connectivity . . . 4
2.1.3 Special Graph Types . . . 5
2.1.4 Graph Representations . . . 5
2.2 Spatial Data Concepts . . . 5
2.2.1 Geographic Coordinate Systems . . . 5
2.2.2 Distance Calculations . . . 6
2.2.3 Speed Constraints and Reachability . . . 6
2.3 Notation Index . . . 6
2.4 Graph Construction Overview . . . 7

**3 GRAPH FORMULATION** 8

3.1 Introduction . . . 8
3.2 Position Reachability Graph . . . 8
3.2.1 Intuition . . . 8
3.2.2 Formal Definitions . . . 8
3.2.3 Graph Properties . . . 9
3.2.4 Extended Reachability with GPS Jitter Tolerance . . . 9
3.2.5 Handover Graph Extension . . . 10

3.3 Mathematical Properties and Theorems . . . . . 10
3.4 Mapping to Standard Graph Problems . . . . . 11
3.5 Handover Detection Flow . . . . . 11
3.6 Summary of Key Formulas . . . . . 12

**4 ALGORITHM DESIGN AND SOLUTION APPROACH 13**
4.1 Introduction . . . . . 13
4.2 Single Vehicle Detection Algorithm . . . . . 13
4.2.1 Algorithm Overview . . . . . 13
4.2.2 Preprocessing Phase . . . . . 14
4.2.3 Initial Clustering Phase . . . . . 14
4.2.4 Handover-Based Cluster Merging . . . . . 16
4.2.5 Decision Logic . . . . . 17
4.2.6 Complete Algorithm . . . . . 18
4.2.7 Correctness Analysis . . . . . 18
4.3 Complexity Analysis . . . . . 18
4.4 Error Handling and Edge Cases . . . . . 19
4.4.1 Input Validation . . . . . 19
4.4.2 Edge Cases . . . . . 19
4.5 Algorithm Decision Flow . . . . . 19

**REFERENCES 19**

# LIST OF TABLES

1.1 Validation challenge: patterns and expected results . . . . . 2

2.1 Index of notation and symbols . . . . . 6

3.1 Mapping our problem to standard graph problems . . . . . 11

4.1 Complexity summary . . . . . 18
4.2 Edge case handling . . . . . 19

# LIST OF FIGURES

1.1 System overview: end-to-end validation pipeline . . . . . 3

2.1 Graph construction pipeline: from raw GPS data to validation result . . . . . 7

3.1 Handover detection decision flow for cluster pair $(C_1, C_2)$ . . . . . 11

4.1 Complete decision flow of the SingleVehicleCheck algorithm . . . . . 20

# CHAPTER 1

# PROBLEM STATEMENT

## 1.1 Introduction

Modern logistics and supply chain systems rely heavily on GPS-based shipment tracking to provide real-time visibility of goods in transit. Every day, millions of GPS position records are generated as shipments move across the globe through trucks, ships, trains, and aircraft. This data forms the backbone of supply chain visibility platforms, enabling businesses to track deliveries, estimate arrival times, optimize routes, and respond to disruptions.

However, the quality of this GPS data is often compromised by various technical and operational issues. Poor data quality leads to incorrect tracking information being displayed to customers, flawed analytics, billing errors, and degraded trust in the tracking system. Therefore, robust validation mechanisms are essential to ensure data integrity before it reaches downstream consumers.

This project focuses on one of the critical data quality problems encountered in production shipment tracking systems: **Multi-Vehicle Tracking Anomaly**—a single shipment being incorrectly tracked by multiple vehicles simultaneously. We propose a unified graph-theoretic framework to model, detect, and resolve these anomalies.

## 1.2 Problem: Shipment Tracked by Multiple Vehicles

### 1.2.1 Problem Description

In a correctly functioning tracking system, a shipment (such as a container, parcel, or pallet) should be associated with exactly one tracking device at any given time. The GPS positions reported for that shipment should represent the movement of a single physical vehicle carrying the goods.

However, in real-world systems, the following erroneous scenarios occur:

**Scenario A: Device Misconfiguration.** Multiple GPS tracking devices are incorrectly assigned to the same shipment ID in the system database. As a result, positions from two or more devices (potentially on different vehicles in different locations) are all recorded under one shipment.

**Scenario B: Data Integration Errors.** When integrating data from multiple carrier partners or legacy systems, shipment IDs may collide or be incorrectly mapped. Two distinct shipments from different sources get merged into a single shipment record.

**Scenario C: Duplicate Device Transmissions.** A malfunctioning device or network issue causes duplicate position records to be created, sometimes with slightly different coordinates or timestamps.

**Scenario D: System Migration Issues.** During system migrations or updates, historical data from old tracking devices may get incorrectly linked with new device data for the same shipment ID.

### 1.2.2 Observable Symptoms

When a shipment is tracked by multiple vehicles, the position history exhibits characteristic patterns:

- **Impossible Teleportation:** Consecutive positions show the shipment at locations that are physically impossible to reach in the elapsed time. For example, a position in Shanghai at 10:00 AM followed by a position in Shenzhen (1,200 km away) at 10:05 AM would require traveling at 14,400 km/h.
- **Oscillating Locations:** The shipment appears to "jump" back and forth between two or more distinct geographic areas, as positions from different vehicles interleave in the timeline.
- **Concurrent Distant Positions:** Multiple positions recorded at nearly the same timestamp show the shipment at vastly different locations simultaneously.

### 1.2.3 Legitimate Exception: Vehicle Handovers

Not all multi-location patterns indicate errors. Legitimate handovers occur when:

- A truck delivers a container to a port, and a ship picks it up.
- An inter-modal transfer happens at a logistics hub.
- A relay system where one truck hands off to another at designated points.

In these cases, there is a feasible transition: the shipment physically moves from one vehicle to another at a common location within a reasonable time window.

### 1.2.4 The Validation Challenge

The system must distinguish between the cases shown in Table 1.1.

Table 1.1: Validation challenge: patterns and expected results

| Pattern | Cause | Expected Result |
|---|---|---|
| All positions reachable in sequence | Single vehicle, normal tracking | VALID |
| Position gaps with feasible handover | Legitimate vehicle transfer | VALID |
| Concurrent positions at distant locations | Multi-device error | INVALID |
| Oscillating positions between two areas | Data integration error | INVALID |
| Few unreachable positions among many valid ones | GPS glitches or outliers | VALID (with tolerance) |

### 1.2.5 Business Impact

Failure to detect multi-vehicle tracking errors causes:

- **Customer Confusion:** Tracking pages show shipments jumping between locations erratically.
- **Incorrect ETAs:** Arrival time predictions become meaningless when location data is inconsistent.
- **Billing Disputes:** Mileage-based billing calculations are corrupted by phantom movements.
- **Operational Blindness:** Supply chain managers cannot trust the tracking data for decision-making.
- **SLA Violations:** False delay alerts or missed actual delays due to corrupted position data.

## 1.3 System Overview

Figure 1.1 illustrates the end-to-end pipeline of the proposed validation framework.

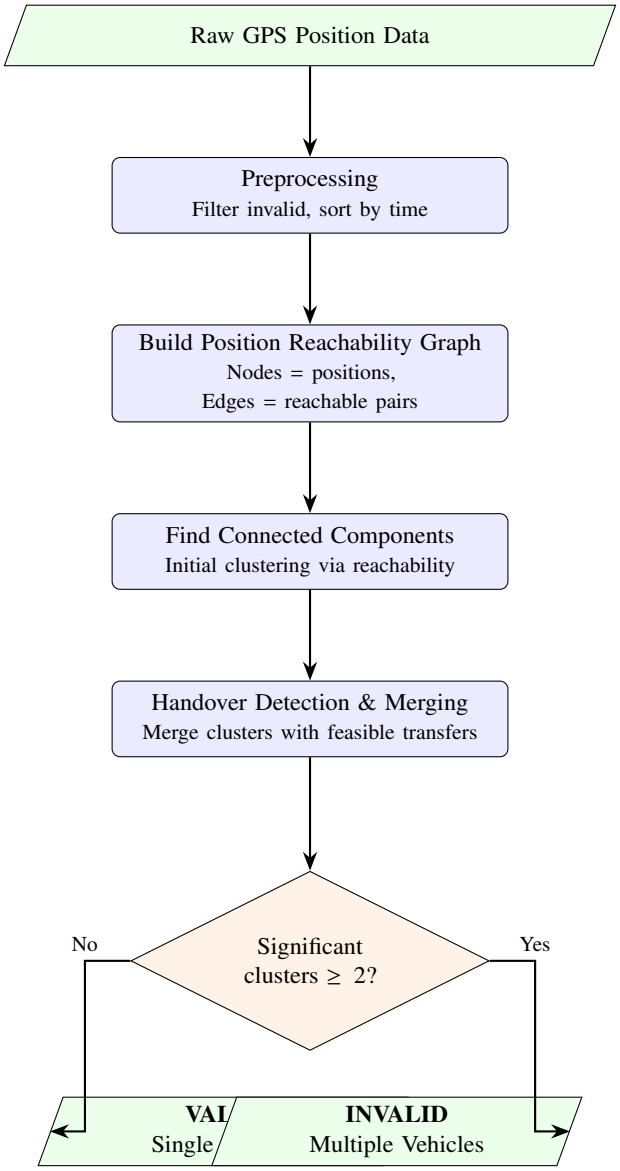


Figure 1.1: System overview: end-to-end validation pipeline

# CHAPTER 2

# GRAPH THEORY BACKGROUND

## 2.1 Graph Theory Fundamentals

### 2.1.1 Basic Definitions

**Definition 1 (Graph)** *A graph $G$ is an ordered pair $G = (V, E)$ where:*

- *$V$ is a set of vertices (also called nodes),*
- *$E$ is a set of edges, where each edge connects two vertices.*

**Definition 2 (Directed Graph)** *A directed graph (or digraph) is a graph where each edge has a direction. An edge from vertex $u$ to vertex $v$ is denoted as $(u, v)$ or $u \to v$, and is distinct from $(v, u)$.*

**Definition 3 (Weighted Graph)** *A weighted graph assigns a numerical value (weight) to each edge. Formally, it includes a weight function $w : E \to \mathbb{R}$ that maps edges to real numbers.*

**Definition 4 (Adjacency)** *Two vertices $u$ and $v$ are adjacent if there exists an edge connecting them. In a directed graph, $u$ is adjacent to $v$ if edge $(u, v)$ exists.*

**Definition 5 (Degree)** *The degree of a vertex $v$, denoted $\deg(v)$, is the number of edges incident to $v$. In directed graphs:*

- ***In-degree:*** *Number of edges coming into $v$*
- ***Out-degree:*** *Number of edges going out from $v$*

### 2.1.2 Paths and Connectivity

**Definition 6 (Path)** *A path in a graph is a sequence of vertices $(v_1, v_2, \ldots, v_k)$ such that each consecutive pair is connected by an edge. The length of the path is $k - 1$ (number of edges).*

**Definition 7 (Connected Graph)** *An undirected graph is connected if there exists a path between every pair of vertices.*

**Definition 8 (Weakly Connected)** *A directed graph is weakly connected if replacing all directed edges with undirected edges results in a connected graph.*

**Definition 9 (Strongly Connected)** *A directed graph is strongly connected if there exists a directed path from every vertex to every other vertex.*

**Definition 10 (Connected Component)** *A connected component of a graph is a maximal subgraph in which every pair of vertices is connected by a path. A graph can have multiple connected components.*

### 2.1.3 Special Graph Types

**Definition 11 (Directed Acyclic Graph (DAG))** *A directed acyclic graph is a directed graph with no directed cycles. That is, there is no sequence of edges that starts and ends at the same vertex.*

**Definition 12 (Bipartite Graph)** *A bipartite graph is a graph whose vertices can be partitioned into two disjoint sets $U$ and $V$ such that every edge connects a vertex in $U$ to a vertex in $V$. No edge connects vertices within the same set.*

**Definition 13 (Complete Graph)** *A complete graph $K_n$ is a graph with $n$ vertices where every pair of vertices is connected by an edge.*

**Definition 14 (Tree)** *A tree is a connected acyclic graph. A tree with $n$ vertices has exactly $n - 1$ edges.*

### 2.1.4 Graph Representations

**Adjacency Matrix.** For a graph with $n$ vertices, the adjacency matrix $A$ is an $n \times n$ matrix where:

- $A[i][j] = 1$ if edge $(i, j)$ exists,
- $A[i][j] = 0$ otherwise.

For weighted graphs, $A[i][j]$ contains the edge weight instead of 1.
Space complexity: $O(n^2)$. Edge lookup: $O(1)$. Finding all neighbors: $O(n)$.

**Adjacency List.** Each vertex maintains a list of its adjacent vertices.
Space complexity: $O(n + m)$ where $m$ is the number of edges. Edge lookup: $O(\text{degree})$. Finding all neighbors: $O(\text{degree})$.

**Implicit Representation.** For some graphs, edges are not stored explicitly but computed on demand using a predicate function. This is memory-efficient when the graph is dense or the edge predicate is simple. Our solution uses implicit representation where edges are computed via the reachability predicate.

## 2.2 Spatial Data Concepts

### 2.2.1 Geographic Coordinate Systems

**Latitude and Longitude.**

- **Latitude** ($\phi$): Angular distance north or south of the equator, ranging from $-90$ to $+90$ degrees.
- **Longitude** ($\lambda$): Angular distance east or west of the Prime Meridian, ranging from $-180$ to $+180$ degrees.

**Coordinate Conventions.**

- Standard: (latitude, longitude)—used by most mapping applications.
- GeoJSON: (longitude, latitude)—used in GeoJSON format and some APIs.

This inconsistency is a root cause of coordinate swap errors.

### 2.2.2 Distance Calculations

**Haversine Formula.** Calculates the great-circle distance between two points on a sphere given their latitudes and longitudes:

$$\text{haversine}(p_1, p_2): \quad R = 6371 \text{ km (Earth's radius)}$$

$$\Delta\phi = p_2.\text{lat} - p_1.\text{lat}, \quad \Delta\lambda = p_2.\text{lon} - p_1.\text{lon}$$

$$a = \sin^2(\Delta\phi/2) + \cos(p_1.\text{lat}) \cdot \cos(p_2.\text{lat}) \cdot \sin^2(\Delta\lambda/2)$$

$$c = 2 \cdot \arcsin(\sqrt{a}), \quad \text{distance} = R \cdot c$$

Accuracy: Assumes spherical Earth; error less than 0.3% compared to ellipsoidal calculations.

**Vincenty Formula.** More accurate distance calculation using ellipsoidal Earth model. Computationally more expensive but provides millimeter accuracy. For our application, Haversine provides sufficient accuracy with better performance.

### 2.2.3 Speed Constraints and Reachability

We use 250 km/h (69.4 m/s) as a conservative upper bound for ground transport, allowing margin for GPS timestamp inaccuracies and brief high-speed segments.

**Reachability Predicate.** Position $p_2$ is reachable from $p_1$ if:

$$\text{distance}(p_1, p_2) \leq \text{MAX_SPEED} \times (p_2.\text{time} - p_1.\text{time})$$

This forms the basis for edge creation in our reachability graph.

## 2.3 Notation Index

Table 2.1 summarizes the mathematical notation used throughout this report.

Table 2.1: Index of notation and symbols

| Symbol | Description |
|---|---|
| $G = (V, E)$ | Graph with vertex set $V$ and edge set $E$ |
| $p_i = (lat, lon, t)$ | Position tuple: latitude, longitude, timestamp |
| $P = \{p_1, \ldots, p_n\}$ | Position sequence sorted by timestamp |
| $V_{\max}$ | Maximum feasible speed (69.4 m/s = 250 km/h) |
| $\text{haversine}(p_i, p_j)$ | Great-circle distance between two positions |
| $\text{reachable}(p_i, p_j)$ | Reachability predicate: dist $\leq V_{\max} \cdot \Delta t$ |
| $C_k$ | Cluster (connected component) of positions |
| $\mathcal{C} = \{C_1, \ldots, C_m\}$ | Set of all clusters |
| $\tau$ | Jitter tolerance window (1800 s = 30 min) |
| $W(p_j, C, \tau)$ | Jitter window: recent positions within $\tau$ seconds |
| $T_{\text{HANDOVER}}$ | Handover time window (28800 s = 8 hours) |
| $\text{first}_k(C)$, $\text{last}_k(C)$ | First/last $k$ boundary positions of cluster $C$ |
| $H = (\mathcal{C}, E_H)$ | Handover Graph over clusters |
| $\alpha$ | Minimum cluster percentage threshold (0.10) |
| $s_{\min}$ | Minimum cluster size threshold (5) |
| $R$ | Earth's radius (6,371,000 m) |

## 2.4 Graph Construction Overview

Figure 2.1 illustrates how raw GPS data is transformed into graph structures for analysis.

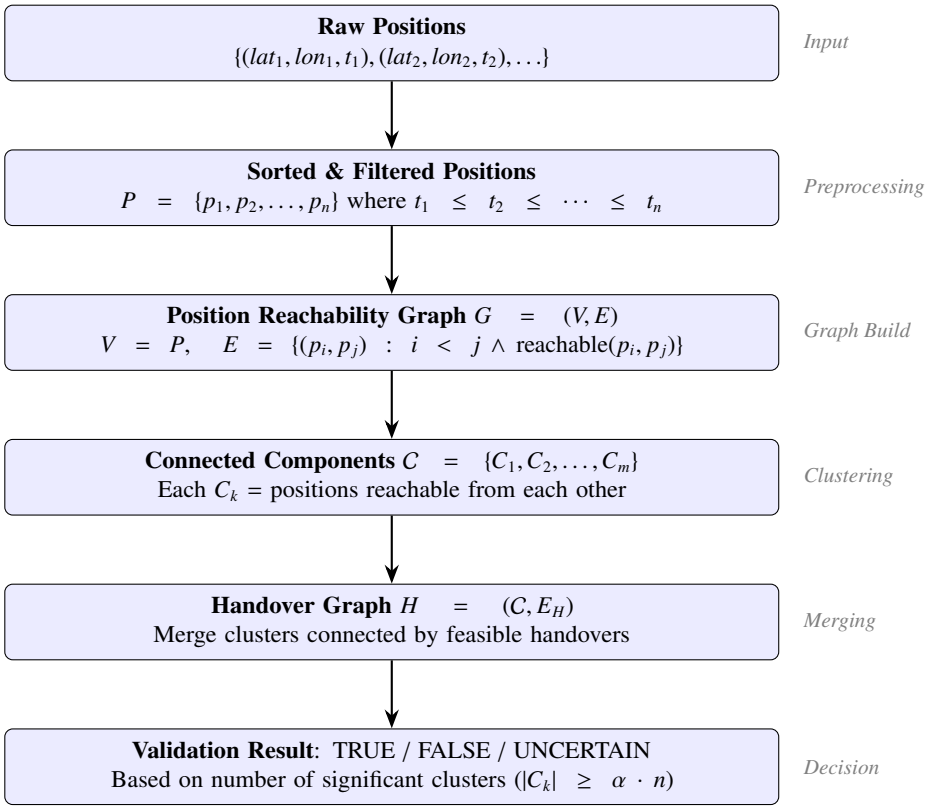


Figure 2.1: Graph construction pipeline: from raw GPS data to validation result

# CHAPTER 3

# GRAPH FORMULATION

## 3.1 Introduction

This chapter presents the formal graph-theoretic models for our shipment tracking validation problems. We transform raw GPS position data into structured graph representations that capture the essential relationships between data points. This abstraction enables us to leverage well-established graph algorithms and provides mathematical rigor to our validation approach.

The key insight is that the problems can be viewed through the lens of connectivity: Are all positions connected through feasible physical movements?

## 3.2 Position Reachability Graph

### 3.2.1 Intuition

Consider a shipment's position history as a sequence of events. If a single vehicle is tracking the shipment, then each position should be "reachable" from previous positions given the physical constraints of movement. Unreachable positions suggest either GPS errors (if few) or multiple vehicles (if many form distinct patterns).

We model this as a directed graph where:

- Each position is a node.
- An edge exists between two positions if one can physically reach the other.
- Connected components represent groups of positions that could belong to the same vehicle.

### 3.2.2 Formal Definitions

**Definition 15 (Position)** *A position $p$ is a tuple $(lat, lon, t)$ where:*

- *$lat$ is the latitude in decimal degrees, $lat \in [-90, 90]$,*
- *$lon$ is the longitude in decimal degrees, $lon \in [-180, 180]$,*
- *$t$ is the timestamp (Unix epoch or datetime).*

**Definition 16 (Position Sequence)** *A position sequence $P$ for a shipment is an ordered set $P = \{p_1, p_2, \ldots, p_n\}$ sorted by timestamp such that $t_1 \leq t_2 \leq \cdots \leq t_n$.*

**Definition 17 (Reachability Predicate)** *Two positions $p_i$ and $p_j$ (where $t_i < t_j$) satisfy the reachability predicate, denoted* reachable$(p_i, p_j)$*, if and only if*

$$\text{haversine}(p_i, p_j) \leq V_{\max} \cdot (t_j - t_i),$$

*where:*

- haversine$(p_i, p_j)$ *is the great-circle distance in meters,*
- $V_{\max}$ *is the maximum feasible speed (69.4 m/s = 250 km/h),*
- $(t_j - t_i)$ *is the time difference in seconds.*

**Definition 18 (Position Reachability Graph)** *Given a position sequence $P$, the Position Reachability Graph is a directed graph $G = (V, E)$ where:*

- $V = P$ *(each position is a vertex),*
- $E = \{(p_i, p_j) : i < j \wedge \text{reachable}(p_i, p_j)\}$.

*Edges are directed from earlier positions to later positions.*

### 3.2.3 Graph Properties

**Property 3.1 (Directed Acyclic Graph)** *The Position Reachability Graph $G$ is always a DAG.*

*Proof.* By construction, edges only go from positions with earlier timestamps to positions with later timestamps. Since $t_i < t_j$ for any edge $(p_i, p_j)$, there cannot exist a cycle. A cycle would require an edge from a later timestamp back to an earlier timestamp, which is impossible by our edge definition.

**Property 3.2 (Temporal Ordering)** *The topological order of $G$ corresponds to the chronological order of positions.*

**Property 3.3 (Transitivity)** *Reachability is not necessarily transitive. If* reachable$(p_1, p_2)$ *and* reachable$(p_2, p_3)$*, it does not imply* reachable$(p_1, p_3)$.

*Example:* $p_1$ at Shanghai, $t = 0$; $p_2$ at Hangzhou (180 km away), $t = 2$ hours (reachable at 90 km/h); $p_3$ at Ningbo (additional 150 km), $t = 3$ hours (reachable from $p_2$ at 150 km/h). Then $p_1$ to $p_3$ is 330 km in 3 hours = 110 km/h (still reachable). With tighter time constraints, transitivity can fail.

### 3.2.4 Extended Reachability with GPS Jitter Tolerance

Real GPS data contains noise. A position might appear slightly unreachable from its immediate predecessor due to GPS jitter, even though it belongs to the same vehicle. We extend reachability to check multiple recent positions.

**Definition 19 (Jitter Window)** *The jitter window $W(p_j, C, \tau)$ for position $p_j$ with respect to cluster $C$ and time threshold $\tau$ is*

$$W(p_j, C, \tau) = \{p_i \in C : (t_j - t_i) \leq \tau\},$$

*the set of positions in cluster $C$ that are within $\tau$ seconds before $p_j$.*

**Definition 20 (Extended Reachability)** *Position $p_j$ is reachable from cluster $C$ with jitter tolerance if*

$$\text{reachable_extended}(p_j, C) = \exists p_i \in W(p_j, C, \tau) : \text{reachable}(p_i, p_j),$$

*where $\tau = 1800$ seconds (30 minutes) in our implementation.*

**Definition 21 (Extended Position Reachability Graph)** *The Extended Position Reachability Graph $G' = (V, E')$ incorporates jitter tolerance: $V = P$, and*

$$E' = \{(p_i, p_j) : i < j \wedge (\text{reachable}(p_i, p_j) \vee \text{exists_jitter_path}(p_i, p_j))\},$$

*where* exists_jitter_path *checks if $p_j$ can be reached through recent positions.*

### 3.2.5 Handover Graph Extension

To handle legitimate vehicle handovers, we introduce a second-level graph.

**Definition 22 (Cluster)** *A cluster $C$ is a maximal set of positions that form a connected component in the Position Reachability Graph.*

**Definition 23 (Cluster Boundary)** *For a cluster $C$:*

- ***Entry boundary:*** $\text{first}_k(C)$ *= first $k$ positions by timestamp,*
- ***Exit boundary:*** $\text{last}_k(C)$ *= last $k$ positions by timestamp,*

*where $k = 5$ in our implementation.*

**Definition 24 (Handover Predicate)** *Two clusters $C_1$ and $C_2$ (where $C_1$ ends before $C_2$ starts) satisfy the handover predicate if*

$$\text{handover}(C_1, C_2) = \exists p_i \in \text{last}_k(C_1),\ p_j \in \text{first}_k(C_2) : |t_i - t_j| \leq T_{\text{HANDOVER}} \wedge \text{reachable}(p_i, p_j),$$

*where $T_{\text{HANDOVER}} = 28800$ seconds (8 hours).*

**Definition 25 (Handover Graph)** *Given clusters $C = \{C_1, C_2, \ldots, C_m\}$, the Handover Graph is $H = (C, E_H)$ where vertices are clusters and $E_H = \{(C_i, C_j) : \text{handover}(C_i, C_j)\}$.*

**Definition 26 (Merged Cluster Graph)** *The Merged Cluster Graph is the transitive closure of the Handover Graph, where connected clusters are merged into single super-clusters.*

## 3.3 Mathematical Properties and Theorems

**Theorem 3.4 (Single Component Implies Single Vehicle)** *If the Position Reachability Graph $G$ has exactly one weakly connected component, then all positions are consistent with single-vehicle tracking.*

*Proof.* A single connected component means every position is reachable from every other position through some path. Since each edge represents a feasible physical movement, the entire trajectory represents feasible single-vehicle movement.

**Theorem 3.5 (Multiple Large Components Implies Multiple Vehicles)** *If $G$ has two or more connected components, each containing at least 10% of total positions, then multiple vehicles are tracking the shipment.*

*Proof.* Two separate components with significant positions cannot both be GPS noise (which would be small, isolated clusters). The lack of edges between components means no feasible path exists between them, implying physically separate vehicles.

**Theorem 3.6 (Handover Preserves Validity)** *If two components $C_1$ and $C_2$ can be merged via a valid handover, the merged component represents valid single-vehicle tracking (with transfer).*

*Proof.* The handover predicate ensures spatial and temporal proximity at the transfer point, consistent with a physical handover event.

## 3.4 Mapping to Standard Graph Problems

Table 3.1: Mapping our problem to standard graph problems

| **Our Problem** | **Standard Graph Problem** |
|---|---|
| Initial clustering | Connected component finding |
| Cluster merging | Transitive closure |
| Significant cluster detection | Component size analysis |
| Handover detection | Bridge edge identification |

## 3.5 Handover Detection Flow

Figure 3.1 shows the decision process for determining whether two clusters represent a legitimate vehicle handover.

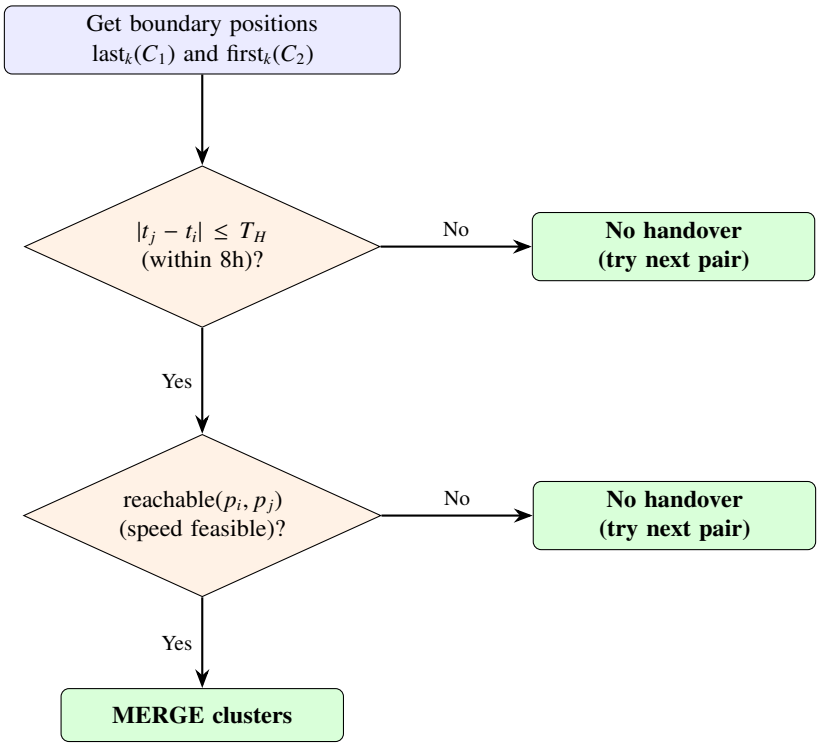


Figure 3.1: Handover detection decision flow for cluster pair $(C_1, C_2)$

## 3.6 Summary of Key Formulas

The following equations summarize the core mathematical foundations of the framework:

$$d(p_i, p_j) = 2R \cdot \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_i)\cos(\phi_j)\sin^2\left(\frac{\Delta\lambda}{2}\right)}\right) \tag{3.1}$$

$$\text{reachable}(p_i, p_j) \iff d(p_i, p_j) \leq V_{\max} \cdot (t_j - t_i) \tag{3.2}$$

$$\text{reachable_ext}(p_j, C) \iff \exists\, p_i \in W(p_j, C, \tau) : \text{reachable}(p_i, p_j) \tag{3.3}$$

$$\text{handover}(C_1, C_2) \iff \exists\, p_i \in \text{last}_k(C_1),\ p_j \in \text{first}_k(C_2) : |t_i - t_j| \leq T_H \ \wedge\ \text{reachable}(p_i, p_j) \tag{3.4}$$

$$\text{significant}(C) \iff |C| \geq \max(s_{\min},\ \alpha \cdot n) \tag{3.5}$$

# CHAPTER 4

# ALGORITHM DESIGN AND SOLUTION APPROACH

## 4.1 Introduction

This chapter presents the detailed algorithms for solving the shipment tracking validation problem. Building on the graph formulations from Chapter 3, we develop step-by-step procedures with pseudocode, analyze their correctness, and evaluate computational complexity.

The algorithms are designed with the following principles:

- **Correctness:** Produce accurate validation results based on graph properties.
- **Efficiency:** Process large position datasets in reasonable time.
- **Robustness:** Handle edge cases and noisy real-world data.
- **Interpretability:** Provide diagnostic information for debugging.

## 4.2 Single Vehicle Detection Algorithm

### 4.2.1 Algorithm Overview

The Single Vehicle Detection algorithm consists of three main phases:

- **INPUT:** Position sequence $P = \{p_1, p_2, \ldots, p_n\}$
- **OUTPUT:** ValidationResult $\in \{\texttt{TRUE}, \texttt{FALSE}, \texttt{UNCERTAIN}\}$

**Phase 1: Preprocessing** — Filter invalid positions; sort by timestamp.
**Phase 2: Clustering** — Build initial clusters using reachability; merge clusters via handover detection.
**Phase 3: Decision** — Analyze cluster structure; return validation result.

### 4.2.2 Preprocessing Phase

**Algorithm 1:** Position Preprocessing

**Input:** Raw position sequence $P$

**Output:** Cleaned, sorted position sequence $P'$

```
P' ← ∅;
foreach position p ∈ P do
    if p.lat or p.lon or p.time is NULL then
        skip;
    if isNaN(p.lat) or isNaN(p.lon) then
        skip;
    if p.lat ∉ [−90, 90] or p.lon ∉ [−180, 180] then
        skip;
    Append p to P';
Sort P' by timestamp ascending;
if |P'| = 0 then
    return empty list;
return P';
```

**Complexity:** Filtering $O(n)$, sorting $O(n \log n)$. Total: $O(n \log n)$.

### 4.2.3 Initial Clustering Phase

The initial clustering phase builds connected components using an online algorithm that processes positions chronologically.

**Algorithm 2:** Build Initial Clusters

**Input:** Sorted position sequence $P = \{p_1, p_2, \ldots, p_n\}$

**Output:** List of clusters $C = \{C_1, C_2, \ldots, C_m\}$

**Constants:** $V_{\max} = 69.4$ m/s, $JITTER_WIN = 1800$ s, $JITTER_CNT = 3$

```
clusters ← ∅;
for i ← 1 to n do
    p ← P[i];
    assigned ← FALSE;
    for j ← |clusters| to 1 do
        C ← clusters[j];
        recent ← GetRecentPositions(C, p.time, JITTER_WIN, JITTER_CNT);
        foreach q ∈ recent do
            if IsReachable(q, p, V_max) then
                Append p to C;
                C.latestTime ← p.time;
                assigned ← TRUE;
                break;
        if assigned then
            break;
    if ¬ assigned then
        Create new cluster with {p} and append to clusters;
return clusters;
```

**Algorithm 3:** IsReachable

**Input:** Positions $p_1, p_2$; maximum speed *maxSpeed*

**Output:** Boolean indicating physical reachability

```
dist ← HaversineDistance(p_1, p_2);
Δt ← |p_2.time − p_1.time|;
if Δt = 0 then
    return dist < 100;
return (dist / Δt) ≤ maxSpeed;
```

**Algorithm 4:** HaversineDistance

**Input:** Positions $p_1 = (lat_1, lon_1)$, $p_2 = (lat_2, lon_2)$

**Output:** Great-circle distance in meters

```
R ← 6,371,000 ;                                   // Earth's radius in meters
Convert lat_1, lon_1, lat_2, lon_2 to radians;
Δφ ← lat_2 − lat_1;
Δλ ← lon_2 − lon_1;
a ← sin²(Δφ/2) + cos(lat_1) · cos(lat_2) · sin²(Δλ/2);
c ← 2 · atan2(√a, √(1 − a));
return R · c;
```

**Algorithm 5:** GetRecentPositions

**Input:** Cluster $C$; current time $t_{\text{cur}}$; time window $\tau$; max count $k$

**Output:** List of recent positions from $C$

```
result ← ∅;
foreach p ∈ C.positions in reverse chronological order do
    if (t_cur − p.time) > τ then
        break;
    Append p to result;
    if |result| ≥ k then
        break;
return result;
```

**Complexity:** Approximately $O(n \cdot m \cdot k)$; for typical cases $O(n)$.

### 4.2.4 Handover-Based Cluster Merging

After initial clustering, we merge clusters that can be connected through feasible handovers.

**Algorithm 6:** Merge Clusters via Handovers

**Input:** List of clusters $C$

**Output:** Merged list of clusters

**Constants:** $T_H = 28800$ s, $b = 5$, $V_{\max} = 69.4$ m/s

```
Sort C by earliest timestamp;
repeat
    merged ← FALSE;
    for i ← 1 to |C| do
        for j ← i + 1 to |C| do
            if CanHandover(C[i], C[j], T_H, b, V_max) then
                MergeClusters(C[i], C[j]);
                Remove C[j];
                merged ← TRUE;
                break;
        if merged then
            break;
until no merges occur;
return C;
```

**Algorithm 7:** CanHandover

**Input:** Clusters $C_1, C_2$; time window $T_H$; boundary size $b$; max speed $V_{\max}$

**Output:** Boolean indicating feasible handover

```
exit ← last b positions of C1 by timestamp;
entry ← first b positions of C2 by timestamp;
foreach p1 ∈ exit do
    foreach p2 ∈ entry do
        if |p2.time − p1.time| ≤ T_H and IsReachable(p1, p2, V_max) then
            return TRUE;
return FALSE;
```

**Algorithm 8:** MergeClusters

**Input:** Clusters $C_1, C_2$

**Output:** $C_1$ with all positions from both clusters

```
Append all positions from C2 to C1;
Sort C1.positions by timestamp;
C1.earliestTime ← min(C1.earliestTime, C2.earliestTime);
C1.latestTime ← max(C1.latestTime, C2.latestTime);
```

**Complexity:** $O(m^3 \cdot b^2)$ with $m$ clusters, $b$ boundary size; $m$ typically small.

### 4.2.5 Decision Logic

**Algorithm 9:** MakeDecision

**Input:** List of clusters $C$; total position count $n$

**Output:** ValidationResult $\in$ {TRUE, FALSE, UNCERTAIN}

**Constants:** $\alpha = 0.10$, $s_{\min} = 5$

```
if |C| = 0 then
    return UNCERTAIN;
if |C| = 1 then
    return TRUE;
threshold ← max(s_min, α × n);
significant ← {C ∈ C : |C.positions| ≥ threshold};
if |significant| = 0 then
    return UNCERTAIN;
else if |significant| = 1 then
    return TRUE;
else
    return FALSE;
```

### 4.2.6 Complete Algorithm

**Algorithm 10:** SingleVehicleCheck — Complete Pipeline

**Input:** Raw position sequence *rawPositions*

**Output:** Result with validation status and diagnostics

```
positions ← PreprocessPositions(rawPositions) ;                    // Alg. 1
if |positions| = 0 then
    return Result(UNCERTAIN, "No valid positions");
if |positions| = 1 then
    return Result(TRUE, "Single position");
clusters ← BuildInitialClusters(positions) ;                       // Alg. 2
clusters ← MergeClustersViaHandovers(clusters) ;                   // Alg. 6
result ← MakeDecision(clusters, |positions|) ;                     // Alg. 9
Build diagnostics: total positions, cluster count, sizes, significant count;
return Result(result, diagnostics);
```

### 4.2.7 Correctness Analysis

**Lemma 4.1** *If all positions belong to a single vehicle, the algorithm returns TRUE.*

*Proof.* A single vehicle's positions will all be mutually reachable (possibly through intermediate positions) given physical speed constraints. The initial clustering will place them in one component, or handover merging will connect any separated clusters. Thus the final result is a single cluster, and the algorithm returns TRUE.

**Lemma 4.2** *If two vehicles are concurrently tracking at distant locations, the algorithm returns FALSE.*

*Proof.* Positions from different vehicles at distant locations cannot satisfy the reachability predicate (impossible speeds required). They will form separate clusters. Since both have significant positions, they will be counted as significant clusters, and the algorithm returns FALSE.

**Theorem 4.3** *The Single Vehicle Check algorithm correctly classifies position sequences.*

*Proof.* Follows from the two lemmas above, plus the UNCERTAIN case for ambiguous situations (small clusters that could be noise or errors).

## 4.3 Complexity Analysis

Table 4.1: Complexity summary

| **Phase** | **Time** | **Space** |
|---|---|---|
| Preprocessing | $O(n \log n)$ | $O(n)$ |
| Initial Clustering | $O(n \cdot m)$ | $O(n)$ |
| Handover Merging | $O(m^3 \cdot b^2)$ | $O(m)$ |
| Decision | $O(m)$ | $O(1)$ |
| **Total** | $O(n \log n + m^3 \cdot b^2)$ | $O(n)$ |

Where $n$ = number of positions, $m$ = number of clusters (typically $m \ll n$), $b$ = boundary size (constant = 5). For typical data with small $m$ (< 10 clusters), the algorithm runs in $O(n \log n)$ time.

## 4.4 Error Handling and Edge Cases

### 4.4.1 Input Validation

The algorithms include robust input validation:

- **Null checks:** Skip positions with null coordinates or timestamps.
- **Range checks:** Validate latitude $[-90, 90]$ and longitude $[-180, 180]$.
- **NaN checks:** Skip positions with NaN values.
- **Empty input:** Return UNCERTAIN for empty position lists.

### 4.4.2 Edge Cases

Table 4.2: Edge case handling

| **Edge Case** | **Handling** |
|---|---|
| No positions | UNCERTAIN |
| Single position | TRUE |
| All positions invalid | UNCERTAIN |
| Zero time difference | Allow 100 m tolerance |

## 4.5 Algorithm Decision Flow

Figure 4.1 presents the complete decision flowchart of the SingleVehicleCheck algorithm, combining all phases from preprocessing through to the final validation result.

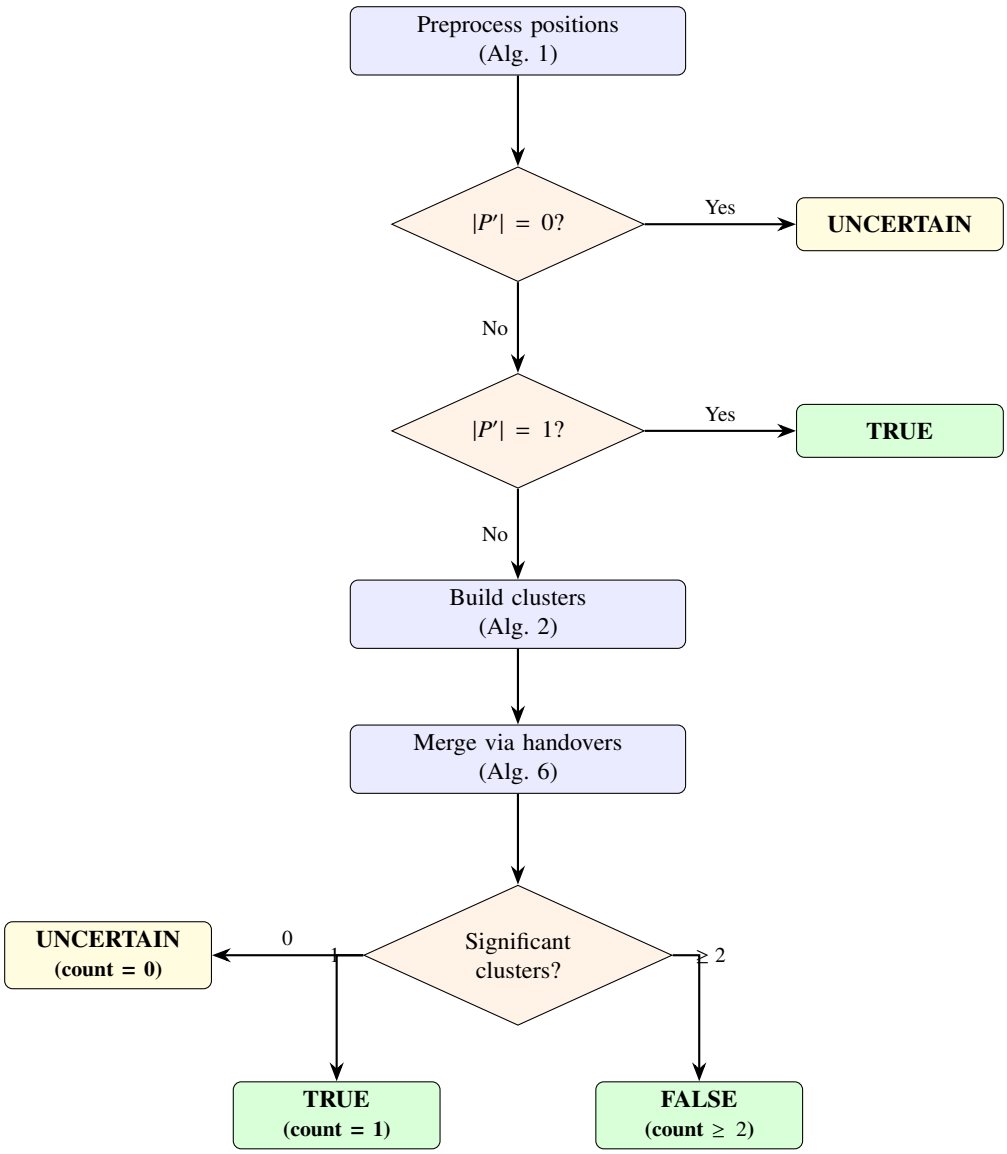


Figure 4.1: Complete decision flow of the SingleVehicleCheck algorithm

# REFERENCES